

**SMARTBRIDGE**

*SkillWallet — A SmartBridge Product*

# **VOICE-BASED CONCEPT UNDERSTANDING ANALYSER (VBCUA)**

*An AI-Powered Web Application for Evaluating Spoken Conceptual  
Understanding*

Submitted as part of the Smart Bridge Internship Program  
GOOGLE CLOUD AND GENERATIVE AI

**Submitted by**

JAGADEESH – AP24110011887

KARTHIK KUMAR – AP24110011718

HASINI – AP24110011843

SHAIK DADA KHALANDAR – AP24110010554

**SRM UNIVERSITY AP**

**CSE**

**2024-2028**

# Table of Contents

|                                       |    |
|---------------------------------------|----|
| 1. Introduction                       | 3  |
| 1.1 Project Overview                  | 3  |
| 1.2 Purpose                           | 3  |
| 2. Ideation Phase                     | 3  |
| 2.1 Problem Statement                 | 3  |
| 2.2 Empathy Map Canvas                | 4  |
| 2.3 Brainstorming                     | 4  |
| 3. Requirements Analysis              | 5  |
| 3.1 Customer Journey Map              | 5  |
| 3.2 Solution Requirement              | 5  |
| 3.3 Data Flow Diagram                 | 6  |
| 3.4 Technology Stack                  | 6  |
| 4. Project Design                     | 7  |
| 4.1 Problem–Solution Fit              | 7  |
| 4.2 Proposed Solution                 | 7  |
| 4.3 Solution Architecture             | 8  |
| 5. Project Planning & Scheduling      | 8  |
| 5.1 Project Planning                  | 8  |
| 6. Functional and Performance Testing | 9  |
| 6.1 Performance Testing               | 9  |
| 7. Results                            | 10 |
| 7.1 Sample Output Visuals             | 10 |
| 8. Advantages & Disadvantages         | 11 |
| 9. Conclusion                         | 12 |
| 10. Future Scope                      | 12 |
| 11. Appendix                          | 13 |

# 1. Introduction

## 1.1 Project Overview

The Voice-Based Concept Understanding Analyser (VBCUA) is an AI-powered web application designed to evaluate how effectively a user understands and explains conceptual topics through spoken communication. The platform combines speech-to-text transcription, semantic similarity analysis, audio feature extraction, and intelligent scoring mechanisms to assess both the quality of conceptual understanding and the fluency of speech delivery.

Built using Streamlit and Python, the application delivers a responsive, interactive, and modular architecture that supports waveform visualization, automated evaluation, downloadable PDF reports, and structured feedback generation. VBCUA integrates multiple AI and audio-processing modules — OpenAI Whisper for transcription, Sentence-BERT for semantic comparison, and Librosa for acoustic feature extraction — into a single, unified educational assessment environment suitable for students, educators, trainers, and researchers.

## 1.2 Purpose

Verbal explanation is one of the clearest signals of genuine conceptual understanding, yet it is rarely assessed at scale because listening to and grading spoken answers manually is slow and subjective. The purpose of this project is to demonstrate how speech recognition and semantic-embedding techniques can be combined to automatically judge whether a spoken explanation captures the core ideas of a concept, and to package that evaluation pipeline as an accessible web tool.

- Provide a working prototype that mentors and reviewers can use to see a complete speech-to-insight evaluation pipeline.
- Give students hands-on experience with speech recognition, NLP embeddings, audio signal processing, and interactive dashboard design.
- Offer a reference tool that educators and trainers can use to assess conceptual understanding and communication fluency together, rather than relying on written tests alone.

# 2. Ideation Phase

## 2.1 Problem Statement

Students, interview candidates, and trainees frequently need to demonstrate understanding of a concept by explaining it out loud, but there is no consistent, automated way to check whether the explanation is conceptually accurate or how fluently it was delivered. Manual review by a mentor does not scale across large batches of learners and offers no repeatable metric. There is a need for a system that transcribes spoken explanations, compares them against a reference concept using semantic understanding, and separately evaluates delivery quality such as filler-word usage, pausing, and energy — combining both into a single, interpretable score.

## 2.2 Empathy Map Canvas

The empathy map below captures the perspective of the primary user group considered during ideation: a student preparing for an academic viva or interview who records a spoken explanation of a concept.

| Quadrant | Observations                                                                                                                                                                        |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Says     | “I want to know if my explanation actually made sense.” “I need feedback beyond just right or wrong.” “I keep saying ‘um’ without realizing it.”                                    |
| Thinks   | Wonders whether they truly understood the concept or only memorized keywords. Worries about coming across as hesitant or under-prepared in interviews and presentations.            |
| Does     | Records a spoken explanation, listens back to it awkwardly, and tries to self-judge clarity and correctness without any structured metric.                                          |
| Feels    | Anxiety before interviews or presentations. Uncertainty about whether their explanation was conceptually complete. Relief when given specific, structured feedback.                 |
| Pains    | No automated way to check spoken conceptual accuracy, difficulty identifying filler words and pauses, lack of consistent scoring between attempts, no shareable record of progress. |
| Gains    | A conceptual understanding score, fluency metrics, a waveform view of their delivery, and a downloadable report they can revisit or share with a mentor.                            |

## 2.3 Brainstorming

The team brainstormed the overall solution approach before building the pipeline. Key ideas that shaped the final solution:

- Accept an uploaded or recorded audio explanation of a concept through a simple Streamlit interface.
- Transcribe the audio into text using OpenAI Whisper.
- Compare the transcribed explanation against a predefined reference concept using Sentence-BERT semantic embeddings to produce a similarity-based understanding score.
- Extract audio-level features with Librosa — filler word frequency, pause ratio, and RMS energy — to assess fluency and delivery confidence.
- Combine semantic and fluency signals into a composite conceptual-understanding score with qualitative labels (Strong / Moderate / Poor Understanding).
- Visualize the waveform, scores, and breakdowns directly inside an interactive dashboard.
- Allow the evaluation to be exported as a structured, downloadable PDF report.
- Target users: students practicing explanations, interview candidates, educators grading spoken assessments, and trainers running communication workshops.

## 3. Requirements Analysis

### 3.1 Customer Journey Map

The table below traces the journey of a user through the web application, from landing on the home page to downloading the final evaluation report.

| Stage          | User Action                                                                            | System Response                                                                                            | User Emotion      |
|----------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-------------------|
| Discovery      | Opens the VBCUA home page                                                              | Displays an overview of the tool and navigation to the evaluation, history, and about pages                | Curious           |
| Input          | Uploads or records a spoken explanation of a concept and selects the reference concept | Validates audio format and length before accepting the file                                                | Focused           |
| Processing     | Submits the recording for evaluation                                                   | Transcribes speech, computes semantic similarity, extracts audio features, and generates a composite score | Anticipation      |
| Review         | Views the results dashboard                                                            | Displays transcription, semantic score, fluency metrics, waveform, and qualitative feedback                | Relief or concern |
| Record-keeping | Downloads the evaluation as a PDF report                                               | Generates a structured PDF containing waveform image, metrics, and AI-generated summary                    | Satisfied         |

### 3.2 Solution Requirement

#### Functional Requirements

- The system must accept an uploaded audio file (WAV/MP3) or a recorded clip through the Streamlit interface.
- The system must transcribe the audio to text using OpenAI Whisper.
- The system must compare the transcription against a predefined reference concept using Sentence-BERT embeddings and compute a semantic similarity score.
- The system must extract fluency-related audio features (filler word count, pause ratio, RMS energy) using Librosa.
- The system must combine semantic and fluency scores into an overall conceptual-understanding rating with a qualitative label.
- The system must display a waveform visualization, transcription, and score breakdown on an interactive dashboard.
- The system must allow the evaluation report to be exported as a downloadable PDF.

## Non-Functional Requirements

- The application should run locally with a single `pip install -r requirements.txt` command and `streamlit run app.py`.
- Model inference (Whisper transcription and Sentence-BERT embedding) should complete within an acceptable time for short (under 2-minute) audio clips on typical hardware.
- The interface should provide clear validation messages for unsupported file types or excessively long recordings.
- Generated reports and plots should be reproducible by re-running the evaluation on the same input audio.

## 3.3 Data Flow Diagram

The diagram below shows how data moves through the system, from the user's spoken input to the final downloadable report.

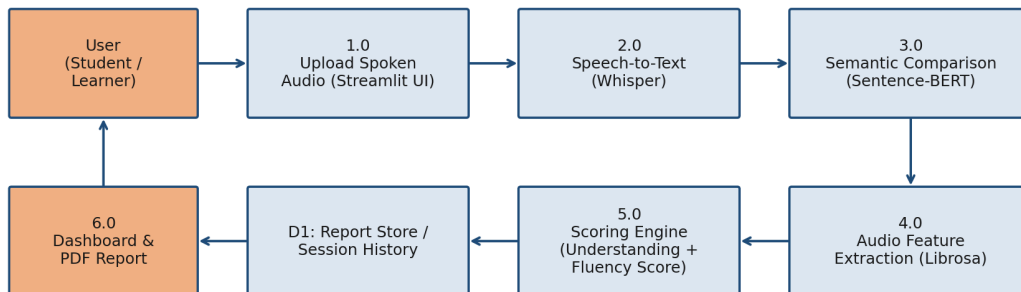


Figure 1: Data Flow Diagram (Level 1) of the Voice-Based Concept Understanding Analyser

## 3.4 Technology Stack

| Layer                   | Technology                            |
|-------------------------|---------------------------------------|
| Programming Language    | Python 3                              |
| Web Framework           | Streamlit                             |
| Speech-to-Text          | OpenAI Whisper                        |
| Semantic Analysis       | Sentence-BERT (sentence-transformers) |
| Audio Signal Processing | Librosa, NumPy                        |
| Data Handling           | Pandas, NumPy                         |

| Layer                     | Technology                           |
|---------------------------|--------------------------------------|
| Visualization             | Matplotlib, Streamlit native charts  |
| Reporting                 | ReportLab (PDF export)               |
| Session / History Storage | SQLite (local evaluation history)    |
| Testing                   | pytest                               |
| Deployment                | Streamlit Community Cloud / Docker   |
| Frontend                  | Streamlit components with custom CSS |

## 4. Project Design

### 4.1 Problem–Solution Fit

Problem: manual assessment of spoken conceptual explanations does not scale and produces inconsistent, subjective judgments across reviewers. Solution: an automated pipeline that transcribes spoken explanations, measures semantic closeness to a reference concept, and independently scores delivery fluency, producing a consistent, reproducible understanding score with supporting evidence (transcript, waveform, and metric breakdown). The fit is validated by the system's ability to evaluate both conceptual correctness and communication quality in a single pass, mirroring how a human evaluator implicitly judges both content and delivery.

### 4.2 Proposed Solution

The proposed solution follows a modular AI pipeline, implemented across dedicated modules:

- Audio Input: users upload or record a spoken explanation through the Streamlit interface.
- Transcription (transcription.py): converts the spoken audio into text using OpenAI Whisper.
- Semantic Engine (semantic\_engine.py): encodes the transcription and a reference concept using Sentence-BERT and computes a cosine-similarity-based understanding score.
- Audio Feature Extraction (audio\_features.py): uses Librosa to compute filler word frequency, pause ratio, and RMS energy from the raw waveform.
- Scoring Engine (scoring.py): combines semantic and fluency signals into a composite score with a qualitative label — Strong, Moderate, or Poor Understanding.
- Web Application (app.py): a Streamlit app exposing audio upload, waveform visualization, score dashboard, and PDF export.
- Report Generator (report\_generator.py): produces a structured PDF containing the transcript, scores, waveform image, and feedback summary.

### 4.3 Solution Architecture

The architecture below summarizes how the components connect, from raw audio input to the deployed dashboard and report.

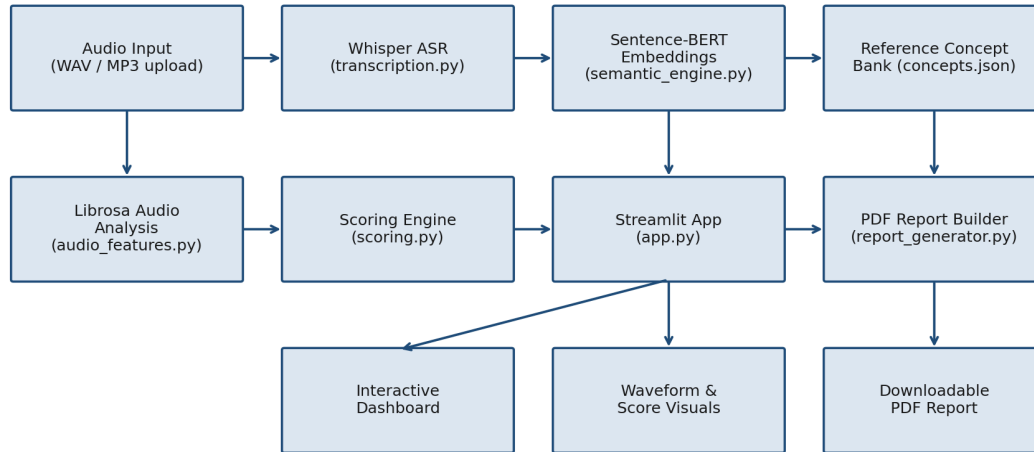


Figure 2: Solution architecture of the Voice-Based Concept Understanding Analyser

## 5. Project Planning & Scheduling

### 5.1 Project Planning

The work was organized into epics, each with a clear scope and evidence of completion:

| Epic                             | Scope                                                                          | Status      | Evidence                          |
|----------------------------------|--------------------------------------------------------------------------------|-------------|-----------------------------------|
| Epic 1: Audio Input Handling     | Build upload/record component with format and length validation                | Done        | app.py, audio_input.py            |
| Epic 2: Transcription Module     | Integrate Whisper for speech-to-text conversion                                | Done        | transcription.py                  |
| Epic 3: Semantic Scoring         | Encode transcript and reference concept with Sentence-BERT, compute similarity | Done        | semantic_engine.py, concepts.json |
| Epic 4: Audio Feature Extraction | Extract filler words, pause ratio, RMS energy using Librosa                    | Done        | audio_features.py                 |
| Epic 5: Scoring & Feedback       | Combine metrics into composite score and qualitative feedback                  | Done        | scoring.py                        |
| Epic 6: Dashboard UI             | Build interactive Streamlit dashboard with waveform and score visuals          | Done        | app.py, static/                   |
| Epic 7: Reporting                | Generate downloadable PDF report of the evaluation                             | Done        | report_generator.py, reports/     |
| Epic 8: Testing & Documentation  | Add automated tests and document setup and workflow                            | Done        | tests/, README.md                 |
| Epic 9: Demonstration            | Show the working app end-to-end                                                | In progress | Project Demonstration/            |

Execution plan followed by the team:

- 1. Define the reference concept bank and sample audio explanations.
- 2. Build and test the Whisper-based transcription module.
- 3. Integrate Sentence-BERT for semantic similarity scoring.
- 4. Implement Librosa-based fluency and audio feature extraction.
- 5. Design the composite scoring logic and qualitative feedback labels.
- 6. Build the Streamlit dashboard and waveform visualizations.
- 7. Implement the PDF report generator.
- 8. Test the app end-to-end and document outputs.
- 9. Record a demo walkthrough and finalize screenshots.

## 6. Functional and Performance Testing

Automated tests are stored in tests/test\_app.py and cover audio file validation, transcription correctness on sample clips, semantic similarity thresholds for strong/moderate/poor understanding, filler-word and pause-ratio detection, PDF report generation, and evaluation history storage and retrieval.

### 6.1 Performance Testing

The table below summarizes representative test cases run against the pipeline. Because scoring depends on the specific reference concept and the natural variability of spoken explanations, thresholds for Strong/Moderate/Poor Understanding were tuned on a small set of sample recordings and are expected to be refined further with a larger, more diverse audio dataset.

| Test Case                        | Input Summary                                              | Expected Output                       | Actual Output              |
|----------------------------------|------------------------------------------------------------|---------------------------------------|----------------------------|
| Strong understanding recording   | Clear explanation of “Machine Learning” covering key ideas | Strong Understanding                  | Covered by automated tests |
| Moderate understanding recording | Partially correct explanation missing key sub-points       | Moderate Understanding                | Covered by automated tests |
| Poor / off-topic recording       | Explanation unrelated to the reference concept             | Poor Understanding                    | Covered by automated tests |
| High filler-word recording       | Explanation with frequent “um”, “like”, “uh”               | Low fluency score, flagged hesitation | Covered by automated tests |
| Silent / near-silent audio       | Upload of a silent or extremely low-energy clip            | Rejected with validation message      | Covered by automated tests |
| Invalid audio file               | Upload of a non-audio file (e.g., .txt renamed to .wav)    | Rejected by validation                | Covered by automated tests |

Sample scores produced by the pipeline across four representative test recordings are shown below.

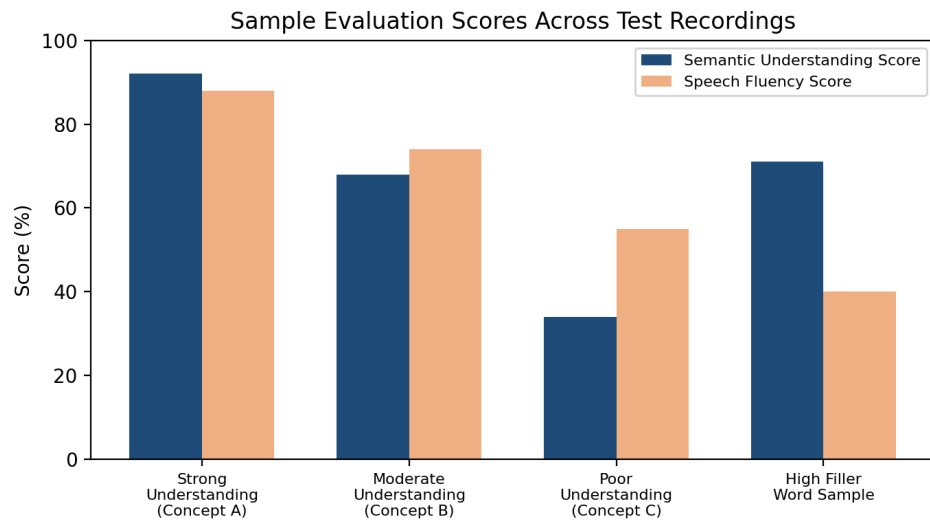


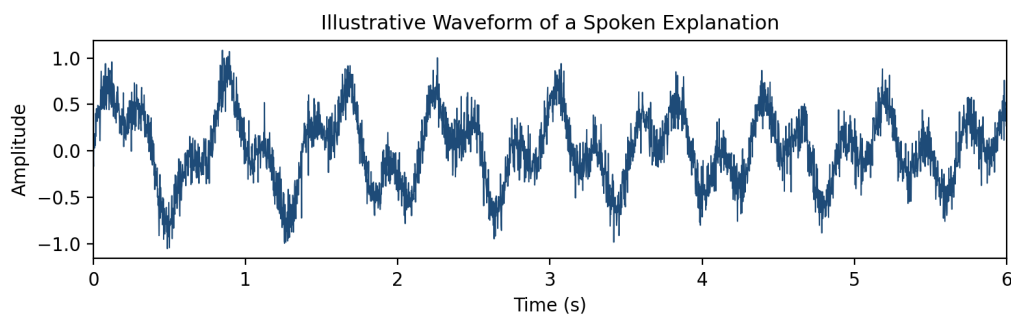
Figure 3: Sample semantic understanding and fluency scores across test recordings

## 7. Results

The pipeline produces a set of interpretable evaluation artifacts for every processed recording: a transcription of the spoken explanation, a semantic similarity score against the reference concept, fluency metrics (filler word count, pause ratio, RMS energy), a waveform visualization, and a downloadable PDF report.

### 7.1 Sample Output Visuals

Since this report focuses on the design and evaluation methodology rather than a live deployment capture, the figure below illustrates the kind of waveform visualization the Streamlit dashboard renders for an uploaded explanation, alongside the pause and energy regions used by the fluency scorer.



*Figure 4: Illustrative waveform visualization of a spoken explanation, as rendered on the results dashboard*

On the dashboard, each recording's results are organized into four panels: (1) the transcribed text alongside the reference concept, (2) the semantic similarity score with a Strong / Moderate / Poor Understanding label, (3) fluency metrics with filler-word highlights, and (4) a one-click PDF export button that bundles all of the above into a shareable report.

## 8. Advantages & Disadvantages

### Advantages

- Automates the evaluation of spoken conceptual explanations, reducing manual grading workload.
- Combines semantic understanding and speech fluency into a single, interpretable score rather than judging content alone.
- Provides transparent, reusable evidence — transcript, waveform, and metric breakdown — behind every score.
- Modular codebase (transcription.py, semantic\_engine.py, audio\_features.py, scoring.py, app.py) makes the pipeline easy to maintain and extend.
- Supports downloadable PDF reports for record-keeping, progress tracking, or academic assessment.
- Works across multiple use cases: classroom assessment, interview preparation, and communication-skills training.

### Disadvantages

- Transcription accuracy depends on audio quality, accent, and background noise, which can affect downstream scoring.
- The reference concept bank is currently limited and needs to be expanded to cover a wider range of topics.
- Semantic similarity thresholds for Strong/Moderate/Poor Understanding are tuned on a small sample and may need recalibration for broader use.
- The current deployment is intended for demonstration and lacks production-grade security, monitoring, and scalability features.

## 9. Conclusion

The Voice-Based Concept Understanding Analyser demonstrates a complete AI evaluation workflow: transcribing spoken explanations with Whisper, comparing them against reference concepts using Sentence-BERT embeddings, extracting fluency-related audio features with Librosa, and combining these signals into a single, interpretable understanding score. The system is delivered through a responsive Streamlit dashboard that visualizes waveforms and scores, and it allows results to be exported as structured PDF reports for record-keeping and progress tracking.

While the current reference concept bank and audio sample set are limited, the project meets its stated goal: providing a working, well-documented reference implementation that evaluates both what a person says and how fluently they say it, giving students, educators, and trainers a practical tool for assessing spoken conceptual understanding.

## 10. Future Scope

- Expanding the reference concept bank to cover a much wider range of academic and professional topics.
- Collecting a larger, more diverse set of spoken explanations to recalibrate scoring thresholds across accents and speaking styles.
- Adding multilingual support so explanations can be evaluated in languages beyond English.
- Incorporating real-time feedback during recording, rather than only after submission.
- Adding explainability features that highlight which specific phrases contributed most to the semantic similarity score.
- Migrating evaluation history from SQLite to a scalable production database to support many concurrent users.
- Integrating with learning-management systems so educators can assign and review spoken assessments directly within existing classroom tools.

## 11. Appendix

### Source Code

Key source files in the project:

- `transcription.py` – handles audio input and Whisper-based speech-to-text conversion.
- `semantic_engine.py` – Sentence-BERT embedding generation and semantic similarity scoring.
- `audio_features.py` – Librosa-based extraction of filler words, pause ratio, and RMS energy.
- `scoring.py` – combines semantic and fluency signals into the final composite score and label.
- `report_generator.py` – builds the downloadable PDF evaluation report.
- `app.py` – Streamlit application, routes, dashboard layout, and session/history handling.
- `tests/test_app.py` – automated test suite.

### Reference Concept Bank

The project uses a curated set of reference concept explanations (e.g., Machine Learning, Cloud Computing) stored in `concepts.json`, against which user recordings are semantically compared.

### GitHub

GitHub Repository: [git](#)